

Aprendizaje por Refuerzo (o Reforzado) (Reinforcement learning (RL))

UMNG

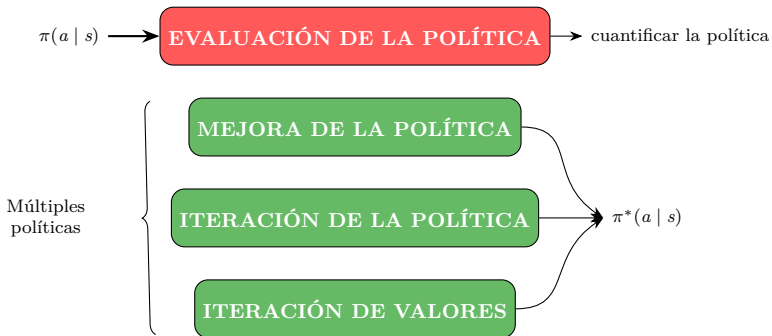
Mayo 2026

Agenda

- 1 Programación Dinámica
 - Evaluación de la política
 - Mejora de la política
 - Iteración de la política
 - Iteración de Valores

- 2 Predicción con Monte Carlo

$$J\pi(a | s)?$$





Evaluación de la política



Política

**Evaluación de la
Política**




**Predicción (valores de la
función estado-valor)**



Evaluación de la política: funcionamiento



Política

**Evaluación de la
Política**

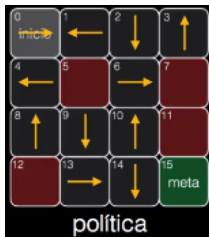


**Predicción (valores de la
función estado-valor)**



Evaluación de la política: funcionamiento

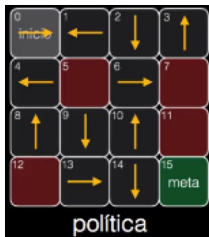
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

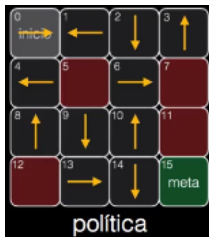
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

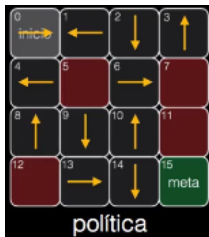
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

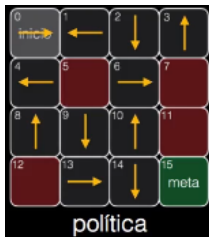
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

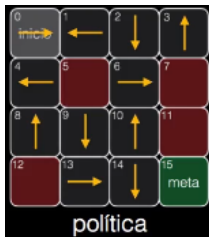
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

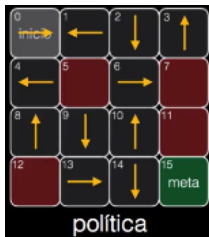
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

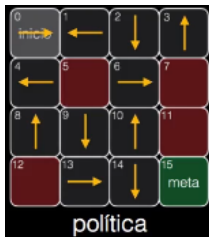
$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]]$$





Evaluación de la política: funcionamiento

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$



La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.
- **Inicializar:**

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.
- **Inicializar:**
 - $v(s) \leftarrow 0$ para todo s (los estados terminales se mantienen en 0).

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.
- **Inicializar:**
 - $v(s) \leftarrow 0$ para todo s (los estados terminales se mantienen en 0).
 - $\Delta \leftarrow 0$ (variación máxima entre iteraciones).

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.
- **Inicializar:**
 - $v(s) \leftarrow 0$ para todo s (los estados terminales se mantienen en 0).
 - $\Delta \leftarrow 0$ (variación máxima entre iteraciones).
- **Iterar:**

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.
- **Inicializar:**
 - $v(s) \leftarrow 0$ para todo s (los estados terminales se mantienen en 0).
 - $\Delta \leftarrow 0$ (variación máxima entre iteraciones).
- **Iterar:**

La evaluación de la política: el algoritmo

- **Entrada:** π , la política a evaluar.
- **Parámetro:** $\theta > 0$, umbral para detener el algoritmo en un número finito de iteraciones.
- **Inicializar:**
 - $v(s) \leftarrow 0$ para todo s (los estados terminales se mantienen en 0).
 - $\Delta \leftarrow 0$ (variación máxima entre iteraciones).
- **Iterar:**

Repetir mientras $\Delta > \theta$:

- 1 Para cada $s \in \mathcal{S}$:

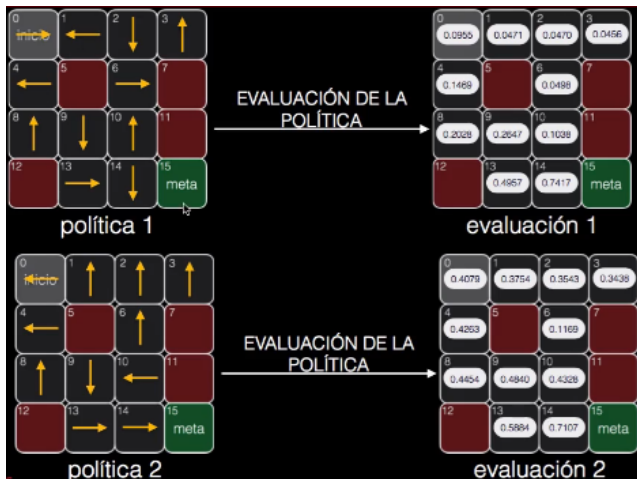
$$\text{temp} \leftarrow v(s)$$

$$v_{k+1}(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

$$\Delta \leftarrow \max(\Delta, |v(s) - \text{temp}|)$$

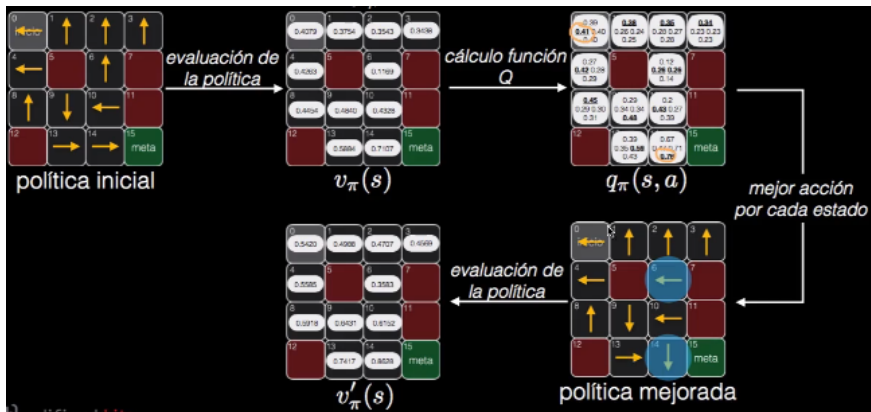


Evaluación de la política: utilidad



Mejora de la política: funcionamiento

$$Q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$



Mejora de la política: el algoritmo en detalle

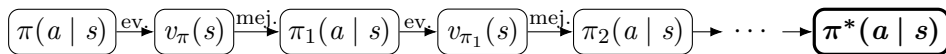
- **Entradas:** $\pi(a \mid s)$ (política a evaluar), $v_\pi(s)$ y γ (el mismo usado en la evaluación de la política).
- Para cada estado s y cada acción a , calcular la **función Q** a partir de la ecuación de Bellman:

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$$

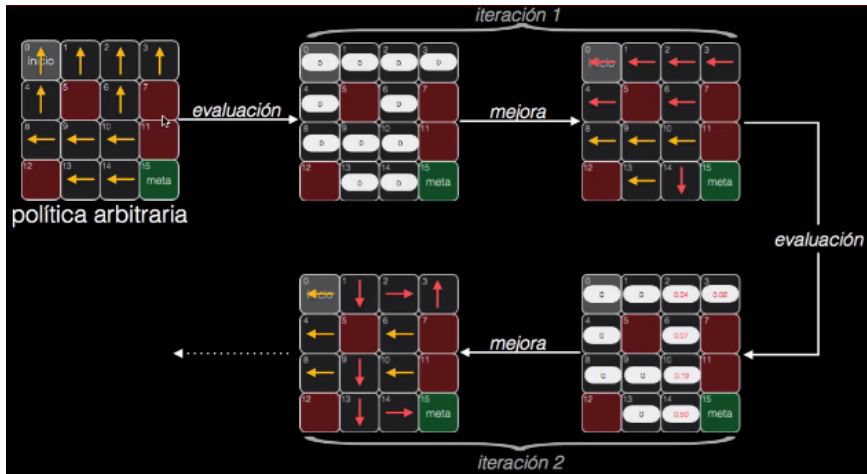
- La **política mejorada** toma, en cada estado, la acción que maximiza q_π :

$$\pi'(a \mid s) = \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$$

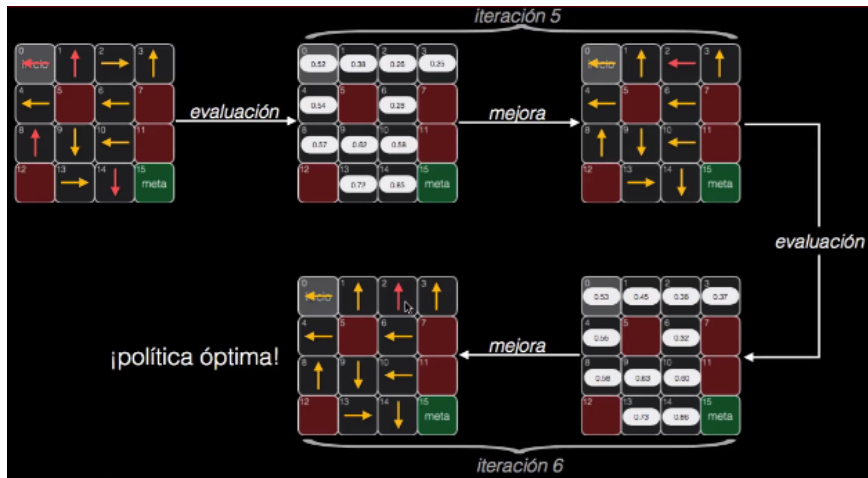
Iteración de la política (idea)



Iteración de la política: funcionamiento



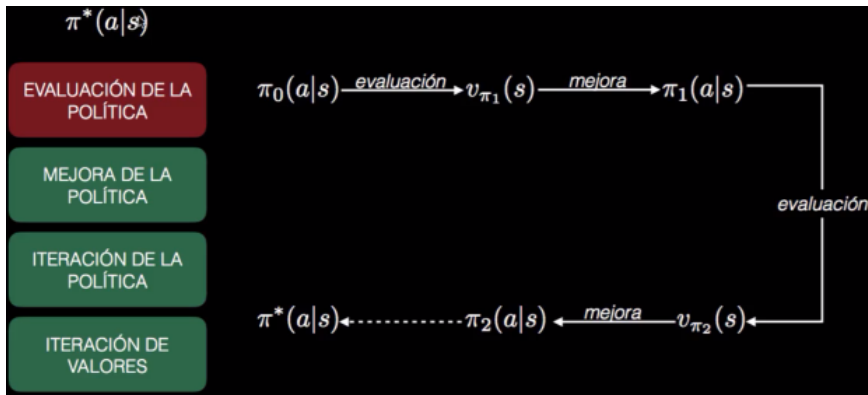
Iteración de la política: funcionamiento



Iteración de la política: el algoritmo

- **Entradas:** política inicial $\pi_0(a \mid s)$, MDP (incl. γ), tolerancia θ .
- **Repetir:**
 - Guardar política_anterior $\leftarrow \pi$
 - Evaluación: obtener $v_\pi(s)$
 - Calcular $q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$
 - Mejora: $\pi(a \mid s) \leftarrow \arg \max_a q_\pi(s, a)$
 - Si $\pi = \text{política_anterior}$ (o cambio $< \theta$), **parar**.

Flujo de Programación Dinámica



Iteración de valores: las ecuaciones

Evaluación:
$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]$$

Mejora:
$$Q_{\pi}(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]$$

$\Downarrow$

Actualización de iteración de valores:

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')]$$

Iteración de valores: el algoritmo

Entradas: función estado–valor inicial $v_0(s)$; MDP completo (incluye γ); tolerancia θ para la convergencia.

Iterar para $k = 0, 1, 2, \dots$:

- Actualización de valores:

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')]$$

- Criterio de parada:

$$\Delta = \max_s |v_{k+1}(s) - v_k(s)|, \quad \text{si } \Delta < \theta \text{ detener.}$$

Política óptima (a partir de v_*):

$$\pi^*(s) = \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]$$

Limitaciones de la Programación Dinámica

Objetivo

$$\pi^*(a|s)$$

Limitaciones de la Programación Dinámica

Objetivo

$$\pi^*(a|s)$$

Componentes de PD

- Evaluación de la política
- Mejora de la política
- Iteración de la política
- Iteración de valores

Limitaciones de la Programación Dinámica

Objetivo

$$\pi^*(a|s)$$

Ecuaciones básicas

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_k(s')]$$

Componentes de PD

- Evaluación de la política
- Mejora de la política
- Iteración de la política
- Iteración de valores

$$Q_{\pi}(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$

Limitaciones de la Programación Dinámica

Objetivo

$$\pi^*(a|s)$$

Ecuaciones básicas

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_k(s')]$$

Componentes de PD

- Evaluación de la política
- Mejora de la política
- Iteración de la política
- Iteración de valores

$$Q_\pi(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma v_\pi(s')]$$

Limitaciones de PD

- **Requiere modelo completo** del entorno: $p(s', r|s, a)$ conocido.
- **Costo computacional alto**: barridos sobre *todos* los estados/acciones en cada iteración.
- **Escalabilidad**: difícil en espacios de estado/acción grandes.
- **Convergencia lenta** si γ es cercano a 1 o la red de transiciones es extensa.

Programación dinámica vs. Monte Carlo

Programación dinámica

$$\{S, A, R, p, \gamma\}$$

- Requiere **modelo** $p(s', r \mid s, a)$.
- Barridos sobre todos los estados/acciones.
- Útil para MDPs pequeños/medianos con modelo conocido.

Monte Carlo

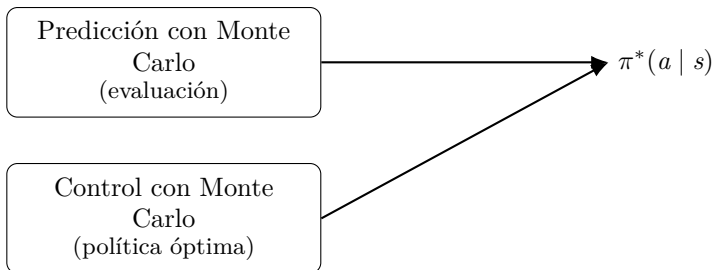
$$\{S, A, R, \gamma\} + \mathcal{D} \text{ (experiencia)}$$

- **No requiere modelo** p .
- Estima valores a partir de episodios ($\mathcal{D}$).
- Convergencia por promediar retornos al final de cada episodio.

Monte Carlo: principio de funcionamiento

$$\hat{v}_{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$



Predicción con Monte Carlo (evaluación de la política)



Predicción con Monte Carlo (evaluación de la política)



Política

$$G = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \quad \gamma = 0,9$$

Predicción con Monte Carlo (evaluación de la política)



Política

$$G = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \quad \gamma = 0,9$$

Episodio	Trayectoria	Retorno (G)
1	0 → 1 → 2 → 6 → 7	0
2	0 → 1 → 2 → 6 → 5	0
3	0 → 1 → 2 → 6 → 10 → 9 → 13 → 14 → 15	0.43
4	0 → 4 → 5	0
5	0 → 4 → 8 → 9 → 10 → 14 → 15	0.53
6	0 → 4 → 8 → 12	0
7	0 → 1 → 2 → 3 → 2 → 6 → 10 → 9 → 8 → 9 → 13 → 14 → 15	0.28
8	0 → 1 → 2 → 6 → 10 → 6 → 7	0
9	0 → 1 → 0 → 4 → 8 → 9 → 5	0
10	0 → 1 → 2 → 3 → 2 → 1 → 5	0

Predicción con Monte Carlo (evaluación de la política)



Política

$$G = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \quad \gamma = 0,9$$

Episodio	Trayectoria	Retorno (G)
1	0 → 1 → 2 → 6 → 7	0
2	0 → 1 → 2 → 6 → 5	0
3	0 → 1 → 2 → 6 → 10 → 9 → 13 → 14 → 15	0.43
4	0 → 4 → 5	0
5	0 → 4 → 8 → 9 → 10 → 14 → 15	0.53
6	0 → 4 → 8 → 12	0
7	0 → 1 → 2 → 3 → 2 → 6 → 10 → 9 → 8 → 9 → 13 → 14 → 15	0.28
8	0 → 1 → 2 → 6 → 10 → 6 → 7	0
9	0 → 1 → 0 → 4 → 8 → 9 → 5	0
10	0 → 1 → 2 → 3 → 2 → 1 → 5	0

$$\tilde{v}_{\pi}(s = 0) = \frac{0+0+0+0,43+0+0,53+0+0,28+0+0}{10} = 0,124$$

iteración de la política en monte carlo: control con monte carlo

Objetivo: obtener una **política óptima** combinando *iterativamente* algoritmos de **evaluación** y **mejora**.

política inicial

$$\pi_0 \xrightarrow{\text{evaluación}} \tilde{q}_{\pi_0} \xrightarrow{\text{mejora}} \pi_1 \xrightarrow{\text{evaluación}} \tilde{q}_{\pi_1} \xrightarrow{\text{mejora}} \dots$$

no conocemos el modelo

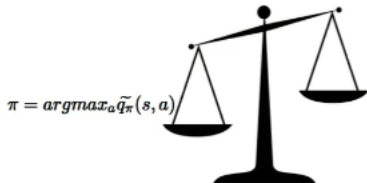
$$(MC \text{ usa muestras/episodios}) \quad \pi \leftarrow \arg \max_a \tilde{q}_{\pi}(s, a)$$

mejora

Idea clave: alternar *predicción* (estimar $\tilde{q}_{\pi}$ con retornos Monte Carlo) y *control* (actualizar π eligiendo acciones con mayor valor estimado). Con suficientes episodios y exploración adecuada (p. ej., ϵ -greedy), el proceso converge a π^* .

Exploración Vs. Explotación

explotación



exploración



Predicción con Monte Carlo: algoritmo de primera visita

Entradas

- π — política a evaluar (genera trayectorias y recompensas).

Inicializar

- $V(s)$ — valor inicial arbitrario para cada estado s .
- $\text{Retornos}(s)$ — lista vacía para guardar retornos observados la **primera** vez que se visita s en cada episodio.

Repetir K episodios

- 1 Generar un episodio completo con π :
 $S_0, A_0, R_1, S_1, \dots, S_T$.
- 2 $G \leftarrow 0$.
- 3 Para $t = T - 1, \dots, 0$:
 - 1 $G \leftarrow \gamma G + R_{t+1}$.
 - 2 Si S_t es **primera visita** en el episodio:
 - 1 $\text{Retornos}(S_t) \leftarrow \text{Retornos}(S_t) \cup \{G\}$.
 - 2 $V(S_t) \leftarrow \text{promedio}(\text{Retornos}(S_t))$.

Notas

- Método **sin modelo**: no requiere $p(s' | s, a)$.
- Primera visita evita contar múltiples veces el mismo estado por episodio.
- Con suficientes episodios, $V(s)$ converge al valor verdadero bajo π .

Predicción con diferencia temporal (TD): idea básica

¿Qué problema resuelve?

Dada una política fija π , queremos estimar la función $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ *sin* esperar a que termine el episodio (a diferencia de Monte Carlo).

Predicción con diferencia temporal (TD): idea básica

¿Qué problema resuelve?

Dada una política fija π , queremos estimar la función $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ *sin* esperar a que termine el episodio (a diferencia de Monte Carlo).

Regla de actualización TD(0)

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \quad (\text{error TD})$$

Predicción con diferencia temporal (TD): idea básica

¿Qué problema resuelve?

Dada una política fija π , queremos estimar la función $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ *sin* esperar a que termine el episodio (a diferencia de Monte Carlo).

Regla de actualización TD(0)

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \quad (\text{error TD})$$

- **Bootstrap:** usa la estimación $V(S_{t+1})$ como objetivo parcial (no el retorno completo).
- **On-line:** actualiza en cada transición $(S_t, A_t, R_{t+1}, S_{t+1})$.
- **Parámetros:** $\alpha \in (0, 1]$ (tasa de aprendizaje), $\gamma \in [0, 1]$ (descuento).
- **Estados terminales:** si S_{t+1} es terminal, tomar $V(S_{t+1}) = 0$.

El algoritmo de predicción con diferencia temporal (TD)

Entrada: π — la política a evaluar.

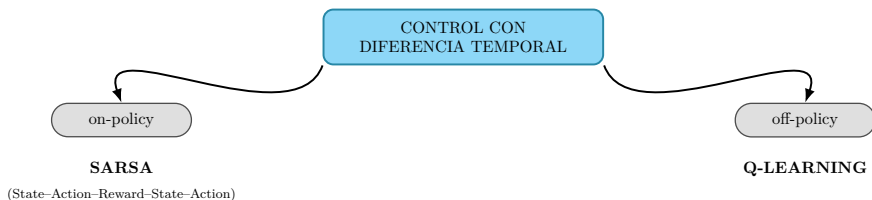
Parámetro: α (tasa de aprendizaje), $0 < \alpha \leq 1$.

Inicializar: $V(s) \leftarrow$ valores arbitrarios para cada estado s ; si s es terminal, $V(s) = 0$. **Repetir K veces y en cada iteración:**

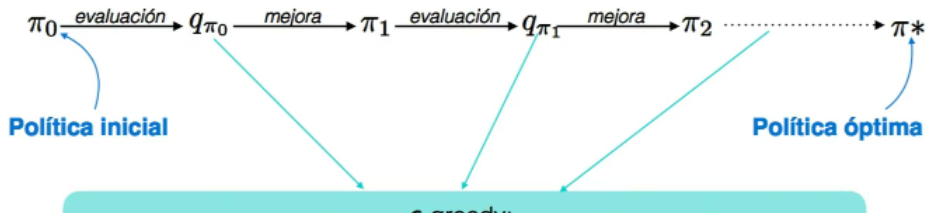
- (a) Generar un episodio usando π .
- (b) Ir al primer estado del episodio S .
- (c) Por cada instante de tiempo t en el episodio:
 - (i) $A \leftarrow$ acción determinada por la política.
 - (ii) Ejecutar A , observar el retorno R y el nuevo estado S' .
 - (iii) Actualizar de forma *incremental* con una sola interacción:

$$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)].$$
 - (iv) $S \leftarrow S'$.
 - (v) Detenerse al llegar a un estado terminal.

Control “on-policy” y control “off-policy”



Control "on policy" con TD



ϵ -greedy:

- Escoger la acción que maximiza con probabilidad $1-\epsilon$
- Escoger cualquier otra acción de forma aleatoria con probabilidad ϵ

SARSA: cálculo de la función acción-valor

$$\underbrace{V(S_t)}_{\text{valor actual}} \leftarrow V(S_t) + \alpha \left(\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{objetivo (target)}} - \underbrace{V(S_t)}_{\text{estimación actual}} \right)$$

$$\Downarrow$$

$$\underbrace{Q(S_t, A_t)}_{\text{par estado-acción}} \leftarrow Q(S_t, A_t) + \alpha \left(\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{objetivo (target)}} - \underbrace{Q(S_t, A_t)}_{\text{estimación actual}} \right)$$



Idea: SARSA es TD(0) aplicado a Q : corrige

$Q(S_t, A_t)$ hacia el *target* $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ usando un paso de aprendizaje α .

Algoritmo SARSA para control on-policy con TD

entradas: política a optimizar π (puede ser arbitraria).

parámetros: α (tasa de aprendizaje), γ (descuento), ε (exploración), todos en $(0, 1]$.

inicializar: $Q(s, a) \leftarrow 0$ para todo par (s, a) ; en estados terminales $Q = 0$.

repetir K veces y en cada iteración:

- (a) generar un episodio usando π .
- (b) ir al primer estado del episodio S y elegir A con ε -greedy sobre Q .
- (c) para cada instante de tiempo t en el episodio:
 - (i) ejecutar A ; observar recompensa R y nuevo estado S' .
 - (ii) elegir la siguiente acción A' en S' con ε -greedy sobre Q (*on-policy*).
 - (iii) actualizar la acción-valor de forma incremental:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)].$$

- (iv) $S \leftarrow S', A \leftarrow A'$.
- (v) detenerse si S es terminal.

SARSA vs. Q-LEARNING

Control con SARSA

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

control con Q-LEARNING

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

sarsa (on-policy): actualiza con la *acción siguiente* A_{t+1} que sigue la misma política (p.ej. ε -greedy).

q-learning (off-policy): actualiza con el *mejor valor* posible en S_{t+1} ($\max_a$), independiente de la política de comportamiento.

Algoritmo Q-LEARNING (off-policy) con TD

Entradas: ninguna (no se necesita una política fija).

Parámetros: α y γ (ambos en $[0, 1]$).

Inicializar: $Q(s, a)$ para cada par estado-acción; los estados terminales inician con $Q = 0$.

Repetir K veces y en cada iteración:

- (a) Generar un episodio usando la política *derivada* de $Q(s, a)$ (por ejemplo, ε -greedy).
- (b) Ir al primer estado del episodio S .
- (c) Por cada instante de tiempo t en el episodio:
 - (i) Tomar la acción A y observar la recompensa R y el nuevo estado S' .
 - (ii) **Actualizar** de forma incremental (TD):

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right].$$

- (iii) $S \leftarrow S'$.
- (iv) Detenerse al llegar a un estado terminal.